UD 5. Arquitectura de Navegación y Comunicación con APIs en React

1. Enrutado con React Router

1.1 *Introducción al Routing en SPAs*

Las **Single Page Applications (SPAs)** son aplicaciones web que cargan una única página HTML y actualizan su contenido dinámicamente sin recargar la página completa. Para gestionar la navegación entre vistas, utilizan un **sistema de routing del lado del cliente.**

React Router es la librería estándar para implementar navegación en React. Permite:

- Definir rutas asociadas a componentes.
- Gestionar parámetros dinámicos.
- Controlar redirecciones y rutas protegidas.
- Crear estructuras de navegación complejas sin recargar la página.

Instalación

```
C:\mi-app>npm install react-router-dom
```

1.2. Configuración básica: BrowserRouter Routes y Route

Para habilitar el enrutado, se envuelve la aplicación con BrowserRouter, que gestiona el historial de navegación mediante la API History del navegador.

Ejemplo básico

/src/App.js

```
// src/App.js

import { BrowserRouter, Routes, Route } from "react-router-dom";
import Home from "./pages/Home";
import About from "./pages/About";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Los conceptos clave son**:**

- **BrowserRouter**: proveedor del contexto de navegación.
- **Routes**: contenedor de rutas.
- **Route**: define una ruta y el componente que debe renderizarse

1.3. Rutas dinámicas: useParams

Las rutas pueden incluir segmentos variables, útiles para mostrar detalles de un elemento (producto, usuario, post…).

Definición de ruta dinámica:

```
<Route path="/users/:id" element={<UserDetail />} />
```

Acceso al parámetro:

```
import { useParams } from "react-router-dom";

function UserDetail() {
  const { id } = useParams();
  return <h2>Usuario: {id}</h2>;
}
```

1.4. Navegación programática: useNavigate

Permite navegar desde funciones o eventos sin usar enlaces <*link*>.

```
import { useNavigate } from "react-router-dom";

function Login() {
  const navigate = useNavigate();

  const handleLogin = () => {
    // lógica de login...
    navigate("/dashboard");
  };

  return <button onClick={handleLogin}>Entrar</button>;
}
```

1.5. Rutas anidadas y Outlet

Las rutas anidadas permiten crear estructuras jerárquicas, como paneles de administración o secciones internas.

Ejemplo:

```
<Route path="/dashboard" element={<Dashboard />}>
  <Route path="profile" element={<Profile />} />
  <Route path="settings" element={<Settings />} />
</Route>
```

Uso de Outlet:

```
import { Outlet } from "react-router-dom";

function Dashboard() {
  return (
    <div>
      <h1>Panel</h1>
      <Outlet />
    </div>
  );
}
```

1.6. Redirecciones con Navigate

Se utiliza para enviar al usuario a otra ruta automáticamente.

```
import { Navigate } from "react-router-dom";

<Route path="/old-route" element={<Navigate to="/new-route"
/>} />
```

1.7. Rutas protegidas

Se emplean para restringir el acceso a ciertas rutas según autenticación o permisos.

Ejemplo de componente ProtectedRoute

```
function ProtectedRoute({ children }) {
  const isAuth = localStorage.getItem("token");

  return isAuth ? children : <Navigate to="/login" />;
}
```

Uso

```
<Route
  path="/admin"
  element={
    <ProtectedRoute>
      <AdminPanel />
    </ProtectedRoute>
  }
/>
```

1.8. Esquema de carpetas y archivos para React Router

```
src/
│
├── main.jsx
├── App.jsx
│
├── pages/
│   ├── Home.jsx
│   ├── About.jsx
│   ├── UserDetail.jsx        // Ruta dinámica
│   ├── Dashboard/
│   │   ├── Dashboard.jsx     // Ruta padre
│   │   ├── Profile.jsx       // Ruta anidada
│   │   └── Settings.jsx      // Ruta anidada
│   └── Login.jsx
│
├── components/
│   ├── Navbar.jsx
│   └── ProtectedRoute.jsx    // Rutas protegidas
│
└── router/
    └── index.jsx             // Configuración central de rutas
```

- **main.js**: Punto de entrada donde se envuelve la app con <BrowserRouter>. (CRA SIEMPRE usa index.js)
- **App.js**: Puede contener layout general o delegar todo a router/index.jsx.
- **pages/**: Carpeta donde viven las vistas asociadas a rutas.
  - Home.js → ruta /
  - About.js → ruta /about
  - UserDetail.js → ruta dinámica /user/:id
  - Dashboard/ → ejemplo de rutas anidadas
  - Dashboard.js→ ruta padre
  - Profile.js y Setting.js → rutas hijas
- **Components**: Componentes reutilizables, como:
  - Navbar.js → navegación con
  - ProtectedRoute.js → wrapper para rutas privadas
- **router/**: Lugar recomendado para centralizar la configuración de rutas.

/src/index.js

```
// src/index.js

import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App";

ReactDOM.createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

/src/App.js

```
// src/App.js

import Navbar from "./components/Navbar";
import Router from "./router";

function App() {
  return (
    <div>
      <Navbar />
      <Router />
    </div>
  );
}

export default App;
```

/src/pages/Home.js

```
// /src/pages/Home.js

import { Link } from "react-router-dom";

export default function Home() {
  return (
    <div style={styles.container}>
      <h1>Bienvenido a la Aplicación</h1>
      <p>Esta es la página de inicio. Usa los enlaces para navegar.</p>

      <nav style={styles.nav}>
```

```
        <Link to="/about" style={styles.link}>Ir a
About</Link>
        <Link to="/users/1" style={styles.link}>Ver
usuario 1</Link>
        <Link to="/dashboard"
style={styles.link}>Ir al Dashboard</Link>
      </nav>
    </div>
  );
}

// Estilos inline simples para mantener el ejemplo
limpio
const styles = {
  container: {
    padding: "2rem",
  },
  nav: {
    marginTop: "1.5rem",
    display: "flex",
    gap: "1rem",
  },
  link: {
    padding: "0.5rem 1rem",
    background: "#1976d2",
    color: "white",
    borderRadius: "4px",
    textDecoration: "none",
  },
};
```

/src/pages/About.js

```
// src/pages/About.js

export default function About() {
  return (
    <div style={styles.container}>
      <h1>Acerca de esta Aplicación</h1>

      <p>
        Esta aplicación está construida con React y
utiliza React Router para
        gestionar la navegación entre diferentes
páginas sin recargar la web.
      </p>

      <p>
        Aquí puedes ver ejemplos de rutas básicas,
```

```
rutas dinámicas, rutas
        anidadas, navegación programática y rutas
protegidas.
      </p>

      <p>
        El objetivo es aprender a estructurar una
SPA moderna y comprender cómo
        funciona el enrutado en el lado del cliente.
      </p>
    </div>
  );
}

// Estilos inline simples para mantener el ejemplo
claro
const styles = {
  container: {
    padding: "2rem",
    lineHeight: "1.6",
  },
};
```

/src/pages/UserDetail.js

```
// src/pages/UserDetail.js

import { useEffect, useState } from "react";
import { useParams } from "react-router-dom";

export default function UserDetail() {
  const { id } = useParams(); // Parámetro de la
ruta dinámica
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // Simulación de petición a API
    const fetchUser = async () => {
      setLoading(true);

      // Simulación de datos
      const fakeUser = {
        id,
        name: "Usuario de ejemplo",
        email: "usuario@example.com",
        role: "Admin",
      };
```

```
      // Simulación de tiempo de carga
      setTimeout(() => {
        setUser(fakeUser);
        setLoading(false);
      }, 800);
    };

    fetchUser();
  }, [id]);

  if (loading) {
    return <p>Cargando datos del usuario...</p>;
  }

  if (!user) {
    return <p>No se encontró el usuario.</p>;
  }

  return (
    <div>
      <h1>Detalle del Usuario</h1>
      <p><strong>ID:</strong> {user.id}</p>
      <p><strong>Nombre:</strong>
{user.name}</p>
      <p><strong>Email:</strong>
{user.email}</p>
      <p><strong>Rol:</strong> {user.role}</p>
    </div>
  );
}
```

/src/pages/Dashboard/Dashboard.js

```
// src/pages/Dashboard/Dashboard.js

import { Link, Outlet } from "react-router-dom";

export default function Dashboard() {
  return (
    <div style={{ padding: "20px" }}>
      <h1>Panel de Control</h1>

      <nav style={{ marginBottom: "20px" }}>
        <ul  style={{  display:  "flex",  gap:  "15px",
listStyle: "none" }}>
          <li>
            <Link to="">Inicio</Link>
          </li>
          <li>
```

```
          <Link to="profile">Perfil</Link>
        </li>
        <li>
          <Link
to="settings">Configuración</Link>
        </li>
      </ul>
    </nav>

    {/* Aquí se renderizarán las rutas anidadas
*/}
    <div style={{ border: "1px solid #ccc",
padding: "20px" }}>
      <Outlet />
    </div>
  </div>
 );
}
```

/src/pages/Dashboard/Profile.js

```
// src/pages/Dashboard/Profile.js

import { useState } from "react";

export default function Profile() {
  const [profile, setProfile] = useState({
    name: "Francisco",
    email: "francisco@example.com",
    role: "Usuario",
  });

  return (
    <div>
      <h2>Perfil del Usuario</h2>
      <p><strong>Nombre:</strong>
{profile.name}</p>
      <p><strong>Email:</strong>
{profile.email}</p>
      <p><strong>Rol:</strong>
{profile.role}</p>
    </div>
  );
}
```

/src/pages/Dashboard/Settings.js

```
// src/pages/Dashboard/Settings.js

import { useState } from "react";

export default function Settings() {
  const [notifications, setNotifications] = useState(true);
  const [theme, setTheme] = useState("light");

  return (
    <div>
      <h2>Configuración</h2>

      <div style={{ marginBottom: "15px" }}>
        <label>
          <input
            type="checkbox"
            checked={notifications}
            onChange={() => setNotifications(!notifications)}
          />
           Activar notificaciones
        </label>
      </div>

      <div>
        <label>
          Tema: 
          <select value={theme} onChange={(e) => setTheme(e.target.value)}>
            <option value="light">Claro</option>
            <option value="dark">Oscuro</option>
          </select>
        </label>
      </div>

      <div style={{ marginTop: "20px" }}>
        <strong>Estado actual:</strong>
        <p>Notificaciones: {notifications ? "Activadas" : "Desactivadas"}</p>
        <p>Tema seleccionado: {theme}</p>
      </div>
    </div>
  );
}
```

/src/pages/Login.js

```
// src/pages/Login.js

import { useState } from "react";
import { useNavigate } from "react-router-dom";

export default function Login() {
  const navigate = useNavigate();

  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");

  const handleLogin = (e) => {
    e.preventDefault();

    // Ejemplo simple de validación
    if (email === "admin@example.com" &&
password === "123456") {
      // Simula login correcto
      navigate("/dashboard");
    } else {
      setError("Credenciales incorrectas");
    }
  };

  return (
    <div style={{ maxWidth: "400px", margin:
"40px auto" }}>
      <h1>Iniciar Sesión</h1>

      <form onSubmit={handleLogin}>
        <div style={{ marginBottom: "15px" }}>
          <label>Email</label>
          <input
            type="email"
            value={email}
            onChange={(e) =>
setEmail(e.target.value)}
            required
            style={{ width: "100%", padding: "8px"
}}
          />
        </div>

        <div style={{ marginBottom: "15px" }}>
          <label>Contraseña</label>
          <input
```

```
          type="password"
          value={password}
          onChange={(e)                          =>
setPassword(e.target.value)}
          required
          style={{ width: "100%", padding: "8px"
}}
        />
      </div>

      {error && (
        <p style={{ color: "red", marginBottom:
"10px" }}>{error}</p>
      )}

      <button
        type="submit"
        style={{
          width: "100%",
          padding: "10px",
          backgroundColor: "#007bff",
          color: "white",
          border: "none",
          cursor: "pointer",
          fontSize: "16px",
        }}
      >
        Entrar
      </button>
    </form>
  </div>
 );
}
```

/src/components/NavBar.js

```
// src/components/NavBar.js

import { NavLink } from "react-router-dom";

export default function Navbar() {
  const isAuthenticated = true;

  const active = { fontWeight: "bold",
textDecoration: "underline" };

  return (
    <nav style={{ padding: "15px", background:
"#222", color: "white" }}>
```

```
      <ul style={{ display: "flex", gap: "20px",
listStyle: "none" }}>
        <li>
          <NavLink to="/" style={({ isActive }) =>
(isActive ? active : null)}>
            Home
          </NavLink>
        </li>

        <li>
          <NavLink to="/about" style={({ isActive })
=> (isActive ? active : null)}>
            About
          </NavLink>
        </li>

        {isAuthenticated && (
          <li>
            <NavLink
              to="/dashboard"
              style={({ isActive }) => (isActive ?
active : null)}
            >
              Dashboard
            </NavLink>
          </li>
        )}

        {!isAuthenticated && (
          <li>
            <NavLink
              to="/login"
              style={({ isActive }) => (isActive ?
active : null)}
            >
              Login
            </NavLink>
          </li>
        )}
      </ul>
    </nav>
  );
}
```

/src/components/ProtectedRoute.js

```
// src/components/ProtectedRoute.js

import { Navigate } from "react-router-dom";

export default function ProtectedRoute({
isAuthenticated = true, children }) {
  if (!isAuthenticated) {
    return <Navigate to="/login" replace />;
  }
  return children;
```

/src/router/index.js

```
// src/router/index.js

import { Routes, Route, Navigate } from "react-
router-dom";

// Páginas
import Home from "../pages/Home";
import About from "../pages/About";
import UserDetail from "../pages/UserDetail";
import Login from "../pages/Login";

// Dashboard (rutas anidadas)
import Dashboard from
"../pages/Dashboard/Dashboard";
import Profile from "../pages/Dashboard/Profile";
import Settings from
"../pages/Dashboard/Settings";

// Componentes
import ProtectedRoute from
"../components/ProtectedRoute";

export default function Router() {
  return (
    <Routes>

      {/* Ruta principal */}
      <Route path="/" element={<Home />} />

      {/* Rutas simples */}
      <Route path="/about" element={<About />}
/>

      {/* Ruta dinámica */}
      <Route path="/users/:id"
```

```
element={<UserDetail />} />

      {/* Login */}
      <Route  path="/login"  element={<Login  />}
/>

      {/* Rutas protegidas */}
      <Route
        path="/dashboard"
        element={
          <ProtectedRoute>
            <Dashboard />
          </ProtectedRoute>
        }
      >
        {/* Rutas anidadas */}
        <Route    path="profile"    element={<Profile
/>} />
        <Route path="settings" element={<Settings
/>} />
      </Route>

      {/* Redirección */}
      <Route                          path="/old-route"
element={<Navigate to="/about" />} />

      {/* Ruta por defecto (404) */}
      <Route  path="*"  element={<h2>Página  no
encontrada</h2>} />

    </Routes>
  );
}
```

2. Integración de funciones CRUD con fichero .json

2.1 Ubicación del fichero .json en la estructura del proyecto

En **Create React App**, tienes *dos ubicaciones posibles* para almacenar un fichero .json, cada una con funciones y limitaciones muy diferentes.

OPCIÓN 1 — Guardar el fichero .json en src/data/

- **Esta es la opción recomendada para tu proyecto**, por estas razones:
- **Acceso directo desde el código:**

Puedes importarlo como si fuera un módulo:

```
import users from "../data/users.json";
```

Esto permite:

- Leer datos de forma inmediata
- Acceder al JSON como un *array real de JavaScript*
- Usarlo dentro del estado (useState) sin fetch

Una limitación importante es que React **NO** permite escribir dentro de /src desde el navegador.

Por lo tanto:

Los cambios CRUD no pueden guardarse físicamente en el fichero .json.

En su lugar, usamos estas alternativas:

- localStorage
- Estado global
- JSON Server *(si quisieras persistencia real)*

OPCIÓN 2 — Accesible vía fetch (simulando una API)

Cargar datos desde un archivo JSON ubicado en public/ usando fetch y mostrarlos en un componente React.

Los pasos a seguir serían:

A. Crea el JSON en public/users.json
B. Componente que lo consume con fetch
C. Usa el componente en tu App

Las rutas correctas que debemos usar:

public/users.json se sirve en /users.json.
Dentro de React, puedes usar **fetch('/users.json')** o **fetch(process.env.PUBLIC_URL + '/users.json')**.

2.2 Carga inicial de datos desde .json

Crea el JSON en public/users.json

```
[
  { "id": 1, "name": "Ada Lovelace", "email": "ada@computing.org",
"role": "admin" },
  { "id": 2, "name": "Alan Turing", "email": "alan@computing.org",
"role": "user" },
  { "id": 3, "name": "Grace Hopper", "email": "grace@navy.mil",
"role": "user" }
]
```

Componente que lo consume con fetch

```
import { useEffect, useState } from 'react';

export default function UsersList() {
  const [users, setUsers] = useState([]);        // datos
  const [loading, setLoading] = useState(true);  // estado de carga
  const [error, setError] = useState(null);      // errores

  useEffect(() => {
    let isMounted = true; // evita setState si el componente se desmonta

    async function loadUsers() {
      try {
        // Opción A (ruta absoluta desde public): '/users.json'
        // Opción B (ruta robusta): process.env.PUBLIC_URL + '/users.json'
        const res = await fetch(`${process.env.PUBLIC_URL}/users.json`, {
          headers: { 'Accept': 'application/json' },
          cache: 'no-store' // útil en desarrollo para evitar caché
        });

        if (!res.ok) {
          // Manejo de errores HTTP (404, 500, etc.)
          throw new Error(`HTTP ${res.status} - ${res.statusText}`);
        }

        const data = await res.json();
```

```
      if (isMounted) {
        setUsers(data);
        setError(null);
      }
    } catch (err) {
      if (isMounted) {
        setError(err.message || 'Error desconocido');
        setUsers([]);
      }
    } finally {
      if (isMounted) setLoading(false);
    }
  }

  loadUsers();

  // Limpieza para evitar actualizar estado tras desmontaje
  return () => {
    isMounted = false;
  };
}, []);

if (loading) return <p>Cargando usuarios…</p>;
if (error)   return <p style={{ color: 'crimson' }}>Error: {error}</p>;
if (!users.length) return <p>No hay usuarios.</p>;

return (
  <section>
    <h2>Usuarios</h2>
    <ul>
      {users.map(u => (
        <li key={u.id}>
          <strong>{u.name}</strong> — {u.email} <em>({u.role})</em>
        </li>
      ))}
    </ul>
  </section>
);
}
```

Usa el componente en tu App

```
import UsersList from './components/UsersList';

export default function App() {
  return (
    <main style={{ fontFamily: 'system-ui, sans-serif', padding: 24
}}>
      <h1>Demo: JSON desde public vía fetch</h1>
      <UsersList />
    </main>
  );
}
```

2.3 Representación del listado de datos (Read)

Los pasos a seguir serían:

1. **Tengo datos en un array**

```
[
  { "id": 1, "name": "Ada Lovelace", "email": "ada@computing.org", "role": "admin" },
  { "id": 2, "name": "Alan Turing", "email": "alan@computing.org", "role": "user" },
  { "id": 3, "name": "Grace Hopper", "email": "grace@navy.mil", "role": "user" }
]
```

2. **Compruebo estados**

```
  // (2) Compruebo estados antes de renderizar el listado
  if (loading) return <p>Cargando usuarios…</p>;
  if (error)   return <p style={{ color: 'crimson' }}>Error: {error}</p>;
  if (!users || users.length === 0) return <p>No hay usuarios.</p>;
```

3. **Recorro el array con map() y Renderizo HTML por cada elemento**

```
  // (3) Recorro el array con map() y Renderizo HTML (lista semántica)
  return (
    <section aria-labelledby="users-title">
      <h2 id="users-title">Usuarios</h2>
      <ul className="users">
        {users.map(user => (
          <li key={user.id} className="user-item">
                <strong>{user.name}</strong> — {user.email} <em>({user.role})</em>
          </li>
        ))}
      </ul>
    </section>
```

4. Carga + paso de props

```
// src/App.jsx
import { useEffect, useState } from 'react';
import UsersList from './components/UsersList';
import './styles.css';

export default function App() {
  const [users, setUsers]     = useState([]);   // (1) Datos en un array (se llenará)
```

```
  const [loading, setLoading] = useState(true); // (2) Estado: cargando
  const [error, setError]     = useState(null); // (2) Estado: error

  useEffect(() => {
    const controller = new AbortController();

    async function fetchUsers() {
      try {
        const res = await fetch(`${process.env.PUBLIC_URL}/users.json`, {
          cache: 'no-store',
          headers: { 'Accept': 'application/json' },
          signal: controller.signal
        });
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        const data = await res.json();
        setUsers(data);          // (1) Aquí llenamos el array
        setError(null);
      } catch (err) {
          if   (err.name   !==   'AbortError')   setError(err.message   ||   'Error
desconocido');
      } finally {
        setLoading(false);       // (2) Dejamos de cargar
      }
    }

    fetchUsers();
    return () => controller.abort();
  }, []);

  return (
    <main className="container">
      <h1>CRUD — READ: Listado de usuarios</h1>
      {/* Pasamos los estados al componente puro de lectura */}
      <UsersList users={users} loading={loading} error={error} />
    </main>
  );
}
```

Un ejemplo de un fichero llamado /src/components/UsersList.js

```
// src/components/UsersList.jsx
import { useEffect, useState } from 'react';

export default function UsersList() {
  // (1) Tengo datos en un array → se cargarán aquí
  const [users, setUsers] = useState([]);
  // (2) Estados de carga y error
  const [loading, setLoading] = useState(true);
  const [error,   setError]   = useState(null);

  useEffect(() => {
    const controller = new AbortController();
```

```
  async function load() {
    try {
      const res = await fetch(`${process.env.PUBLIC_URL}/users.json`, {
        cache: 'no-store',
        headers: { 'Accept': 'application/json' },
        signal: controller.signal
      });
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      const data = await res.json();
      setUsers(data);          // (1) Datos cargados al array
      setError(null);
    } catch (err) {
      if (err.name !== 'AbortError') setError(err.message || 'Error desconocido');
    } finally {
      setLoading(false);       // (2) Fin de la carga
    }
  }

  load();
  return () => controller.abort();
}, []);

// (2) Compruebo estados
if (loading) return <p>Cargando usuarios…</p>;
if (error)   return <p style={{ color: 'crimson' }}>Error: {error}</p>;
if (users.length === 0) return <p>No hay usuarios.</p>;

// (3) map() + (4) HTML
return (
  <section aria-labelledby="users-title">
    <h2 id="users-title">Usuarios</h2>
    <ul className="users">
      {users.map(user => (
        <li key={user.id} className="user-item">
                    <strong>{user.name}</strong>          —          {user.email}
<em>({user.role})</em>
        </li>
      ))}
    </ul>
  </section>
);
}
```

2.4 Creación de nuevos datos (Create)

1. Tener un array en el estado (users)
2. Crear un formulario con inputs controlados
3. Guardar los valores del formulario en el estado
4. Validar los datos
5. Crear un nuevo objeto (con id)
6. Actualizar el array con inmutabilidad
7. Limpiar el formulario
8. El listado se actualiza solo

//SRC/APP.JS

```
// src/App.js
import { useEffect, useState } from 'react';
import UsersForm from './components/UsersForm';
import UsersList from './components/UsersList';

export default function App() {
  // 1. Tener un array en el estado
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  // Cargar datos iniciales desde el .json
  useEffect(() => {
    async function loadUsers() {
      try {
        const res = await fetch(`${process.env.PUBLIC_URL}/users.json`);
        if (!res.ok) throw new Error('Error cargando datos');
        const data = await res.json();
        setUsers(data);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    }

    loadUsers();
  }, []);

  // CREATE: añadir nuevo usuario
  function addUser(userData) {
    // 5. Crear nuevo objeto con id
    const newUser = {
      id: Date.now(),  // suficiente para prácticas
      ...userData
    };

    // 6. Actualizar el array con inmutabilidad
    setUsers(prevUsers => [...prevUsers, newUser]);
  }

  return (
    <main style={{ padding: 24 }}>
      <h1>CRUD con JSON — CREATE + READ</h1>

      <UsersForm onCreate={addUser} />

      <hr />

      {loading && <p>Cargando usuarios…</p>}
      {error && <p style={{ color: 'crimson' }}>Error: {error}</p>}

      {!loading && !error && <UsersList users={users} />}
    </main>
  );
```

```
}
```

/SRC/COMPONENTS/USERFORM.JS

```
// src/components/UsersForm.js
import { useState } from 'react';

export default function UsersForm({ onCreate }) {
  // 3. Estado del formulario
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    role: 'user'
  });

  const [error, setError] = useState(null);

  function handleChange(e) {
    const { name, value } = e.target;
    setFormData(prev => ({
      ...prev,
      [name]: value
    }));
  }

  // 4. Validar datos
  function isValid() {
    if (!formData.name.trim()) return 'El nombre es obligatorio';
    if (!formData.email.trim()) return 'El email es obligatorio';
    return null;
  }

  function handleSubmit(e) {
    e.preventDefault();

    const validationError = isValid();
    if (validationError) {
      setError(validationError);
      return;
    }

    // Crear (enviamos los datos al padre)
    onCreate({
      name: formData.name.trim(),
      email: formData.email.trim(),
      role: formData.role
    });

    // 7. Limpiar formulario
    setFormData({ name: '', email: '', role: 'user' });
    setError(null);
  }

  return (
    <section>
      <h2>Crear nuevo usuario</h2>

      <form onSubmit={handleSubmit}>
```

```
      <div>
        <label>Nombre</label>
        <input
          type="text"
          name="name"
          value={formData.name}
          onChange={handleChange}
        />
      </div>

      <div>
        <label>Email</label>
        <input
          type="email"
          name="email"
          value={formData.email}
          onChange={handleChange}
        />
      </div>

      <div>
        <label>Rol</label>
        <select
          name="role"
          value={formData.role}
          onChange={handleChange}
        >
          <option value="user">user</option>
          <option value="admin">admin</option>
        </select>
      </div>

      {error && <p style={{ color: 'crimson' }}>{error}</p>}

      <button type="submit">Crear</button>
    </form>
  </section>
 );
}
```

2.5 Edición de datos existentes (Update)

1. Tener los datos en el estado (users)
2. Seleccionar qué elemento editar (por id)
3. Clonar los datos del elemento en un formulario controlado
4. Editar en el formulario (estado local)
5. Validar
6. Construir el objeto actualizado (mismo id)
7. Actualizar el array con inmutabilidad
8. Cerrar el modo edición (o limpiar formulario)
9. Re-render automático muestra los cambios

/SRC/APP.JS

```
/src/App.js
import { useEffect, useState } from 'react';
import UsersList from './components/UsersList';

export default function App() {
  const [users, setUsers]   = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError]     = useState(null);

  // Carga inicial desde public/users.json (solo lectura)
  useEffect(() => {
    (async () => {
      try {
        const res = await fetch(`${process.env.PUBLIC_URL}/users.json`, { cache:
'no-store' });
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        setUsers(await res.json());
      } catch (err) {
        setError(err.message || 'Error cargando datos');
      } finally {
        setLoading(false);
      }
    })();
  }, []);

  // UPDATE: función para aplicar cambios (inmutabilidad)
  function updateUser(updated) {
    setUsers(prev =>
      prev.map(u => (u.id === updated.id ? { ...u, ...updated } : u))
    );
  }

  return (
    <main style={{ fontFamily: 'system-ui, sans-serif', padding: 24, maxWidth:
900, margin: '0 auto' }}>
      <h1>CRUD — UPDATE (edición inline)</h1>
      {loading && <p>Cargando…</p>}
      {error && <p style={{ color: 'crimson' }}>Error: {error}</p>}
      {!loading && !error && (
        <UsersList users={users} onUpdate={updateUser} />
      )}
    </main>
  );
}
```

SRC/COMPONENTS/USERLIST.JS

```
src/components/UsersList.js
import { useState } from 'react';
import UserRow from './UserRow';

export default function UsersList({ users, onUpdate }) {
  // id del usuario que está en edición (o null)
  const [editingId, setEditingId] = useState(null);

  if (!users.length) return <p>No hay usuarios.</p>;
```

```
  return (
    <section aria-labelledby="users-title">
      <h2 id="users-title">Usuarios</h2>

      <div style={{ overflowX: 'auto' }}>
        <table>
          <thead>
            <tr>
              <th>Nombre</th>
              <th>Email</th>
              <th>Rol</th>
              <th aria-label="acciones"></th>
            </tr>
          </thead>
          <tbody>
            {users.map(user => (
              <UserRow
                key={user.id}
                user={user}
                isEditing={editingId === user.id}
                onEdit={() => setEditingId(user.id)}
                onCancel={() => setEditingId(null)}
                onSave={(updatedData) => {
                  onUpdate(updatedData);  // aplica cambios en el estado superior
                  setEditingId(null);     // cierra edición
                }}
              />
            ))}
          </tbody>
        </table>
      </div>
    </section>
  );
}
```

SRC/COMPONENTS/USERROW.JS

```
// src/components/UserRow.js
import { useEffect, useState } from 'react';

export default function UserRow({ user, isEditing, onEdit, onCancel, onSave }) {
  // Copia editable local (formulario controlado)
  const [form, setForm] = useState(user);
  const [errors, setErrors] = useState({});

  // Cuando cambia el registro en edición, re-sincroniza el formulario
  useEffect(() => {
    if (isEditing) setForm(user);
  }, [isEditing, user]);

  function handleChange(e) {
    const { name, value } = e.target;
    setForm(prev => ({ ...prev, [name]: value }));
  }

  function validate(values) {
    const err = {};
```

```
  if (!values.name?.trim()) err.name = 'El nombre es obligatorio';
  if (!values.email?.trim()) err.email = 'El email es obligatorio';
  return err;
}

function handleSave(e) {
  e.preventDefault();
  const v = validate(form);
  setErrors(v);
  if (Object.keys(v).length > 0) return;

  // Mantenemos el mismo id
  onSave({ ...form, id: user.id });
}

if (!isEditing) {
  // Modo lectura
  return (
    <tr>
      <td>{user.name}</td>
      <td><a href={`mailto:${user.email}`}>{user.email}</a></td>
      <td>{user.role}</td>
      <td>
        <button type="button" onClick={onEdit}>Editar</button>
      </td>
    </tr>
  );
}

// Modo edición inline
return (
  <tr>
    <td>
      <input
        name="name"
        type="text"
        value={form.name}
        onChange={handleChange}
        aria-invalid={Boolean(errors.name)}
        aria-describedby={errors.name ? `err-name-${user.id}` : undefined}
      />
      {errors.name && (
        <div id={`err-name-${user.id}`} style={{ color: 'crimson', fontSize: 12
}}>
          {errors.name}
        </div>
      )}
    </td>

    <td>
      <input
        name="email"
        type="email"
        value={form.email}
        onChange={handleChange}
        aria-invalid={Boolean(errors.email)}
        aria-describedby={errors.email ? `err-email-${user.id}` : undefined}
```

```
      />
      {errors.email && (
        <div id={`err-email-${user.id}`} style={{ color: 'crimson', fontSize: 12
}}>
         {errors.email}
        </div>
      )}
    </td>

    <td>
      <select name="role" value={form.role} onChange={handleChange}>
        <option value="user">user</option>
        <option value="admin">admin</option>
      </select>
    </td>

    <td>
      <div style={{ display: 'flex', gap: 8 }}>
        <button type="button" onClick={handleSave}>Guardar</button>
        <button type="button" onClick={onCancel}>Cancelar</button>
      </div>
    </td>
  </tr>
 );
}
```

2.6 Eliminación de datos (Delete)

1. Identificar **qué elemento eliminar**
2. Pedir **confirmación**
3. **Actualizar el estado** eliminándolo
4. React **re-renderiza automáticamente** el listado

SRC/APP.JS

```
//src/App.js
import { useEffect, useState } from 'react';
import UsersForm from './components/UsersForm';
import UsersList from './components/UsersList';

export default function App() {
  // Estado principal
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error,   setError]   = useState(null);

  // Carga inicial desde public/users.json
  useEffect(() => {
    const controller = new AbortController();

    async function loadUsers() {
      try {
        const res = await fetch(`${process.env.PUBLIC_URL}/users.json`, {
          cache: 'no-store',
          headers: { 'Accept': 'application/json' },
          signal: controller.signal
```

```
      });
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      const data = await res.json();
      setUsers(data);
      setError(null);
    } catch (err) {
      if (err.name !== 'AbortError') setError(err.message || 'Error cargando datos');
    } finally {
      setLoading(false);
    }
  }

  loadUsers();
  return () => controller.abort();
 }, []);

 // CREATE
 function addUser(userData) {
   const id = (window.crypto?.randomUUID?.() ?? String(Date.now()));
   const newUser = { id, ...userData };
   setUsers(prev => [...prev, newUser]); // inmutabilidad
 }

 // UPDATE
 function updateUser(updated) {
   setUsers(prev => prev.map(u => (u.id === updated.id ? { ...u, ...updated } :
u)));
 }

 // DELETE
 function deleteUser(id) {
    const confirmed = window.confirm('¿Seguro que quieres eliminar este
usuario?');
   if (!confirmed) return;
   setUsers(prev => prev.filter(u => u.id !== id));
 }

 return (
    <main style={{ fontFamily: 'system-ui, sans-serif', padding: 24, maxWidth:
960, margin: '0 auto' }}>
     <h1>CRUD con JSON inicial — Create / Read / Update / Delete</h1>

     {/* CREATE */}
     <UsersForm onCreate={addUser} />

     <hr style={{ margin: '24px 0' }} />

     {/* READ + UPDATE + DELETE */}
     {loading && <p>Cargando usuarios…</p>}
     {error && <p style={{ color: 'crimson' }}>Error: {error}</p>}
     {!loading && !error && (
       <UsersList
         users={users}
         onUpdate={updateUser}
         onDelete={deleteUser}
       />
     )}
```

```
    </main>
  );
}
```

SRC/COMPONENTS/USERFORM.JS

```
//src/components/UsersForm.js (CREATE, formulario controlado + validación)
import { useState } from 'react';

export default function UsersForm({ onCreate }) {
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    role: 'user'
  });
  const [errors, setErrors] = useState({});

  function handleChange(e) {
    const { name, value } = e.target;
    setFormData(prev => ({ ...prev, [name]: value }));
  }

  function validate(values) {
    const errs = {};
    if (!values.name.trim()) errs.name = 'El nombre es obligatorio.';
    if (!values.email.trim()) errs.email = 'El email es obligatorio.';
     else if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(values.email)) errs.email =
'Formato de email no válido.';
    if (!values.role.trim()) errs.role = 'El rol es obligatorio.';
    return errs;
  }

  function handleSubmit(e) {
    e.preventDefault();
    const v = validate(formData);
    setErrors(v);
    if (Object.keys(v).length > 0) return;

    onCreate({
      name: formData.name.trim(),
      email: formData.email.trim(),
      role: formData.role
    });

    // limpia el formulario
    setFormData({ name: '', email: '', role: 'user' });
    setErrors({});
  }

  return (
    <section aria-labelledby="create-user-title">
      <h2 id="create-user-title">Crear nuevo usuario</h2>

      <form onSubmit={handleSubmit} noValidate style={{ display: 'grid', gap: 12,
maxWidth: 520 }}>
        <div>
          <label htmlFor="name">Nombre</label>
          <input
```

```
          id="name"
          name="name"
          type="text"
          placeholder="Ada Lovelace"
          value={formData.name}
          onChange={handleChange}
          aria-invalid={Boolean(errors.name)}
          aria-describedby={errors.name ? 'name-error' : undefined}
        />
        {errors.name && <div id="name-error" style={{ color: 'crimson', fontSize:
14 }}>{errors.name}</div>}
      </div>

      <div>
        <label htmlFor="email">Correo electrónico</label>
        <input
          id="email"
          name="email"
          type="email"
          placeholder="ada@computing.org"
          value={formData.email}
          onChange={handleChange}
          aria-invalid={Boolean(errors.email)}
          aria-describedby={errors.email ? 'email-error' : undefined}
        />
        {errors.email && <div id="email-error" style={{ color: 'crimson', fontSize:
14 }}>{errors.email}</div>}
      </div>

      <div>
        <label htmlFor="role">Rol</label>
        <select
          id="role"
          name="role"
          value={formData.role}
          onChange={handleChange}
          aria-invalid={Boolean(errors.role)}
          aria-describedby={errors.role ? 'role-error' : undefined}
        >
          <option value="user">user</option>
          <option value="admin">admin</option>
        </select>
        {errors.role && <div id="role-error" style={{ color: 'crimson', fontSize: 14
}}>{errors.role}</div>}
      </div>

      <div style={{ display: 'flex', gap: 8 }}>
        <button type="submit">Crear</button>
        <button
          type="button"
          onClick={() => {
            setFormData({ name: '', email: '', role: 'user' });
            setErrors({});
          }}
        >
          Limpiar
        </button>
```

```
      </div>
    </form>
  </section>
 );
}
```

SRC/COMPONENTS/USERSLIST.JS

```
//src/components/UsersList.js — (READ + control de edición + DELETE)
import { useState } from 'react';
import UserRow from './UserRow';

export default function UsersList({ users, onUpdate, onDelete }) {
  const [editingId, setEditingId] = useState(null);

  if (!users.length) return <p>No hay usuarios.</p>;

  return (
    <section aria-labelledby="users-title">
      <h2 id="users-title">Usuarios</h2>

      <div style={{ overflowX: 'auto' }}>
        <table>
          <thead>
            <tr>
              <th>Nombre</th>
              <th>Email</th>
              <th>Rol</th>
              <th aria-label="acciones"></th>
            </tr>
          </thead>
          <tbody>
            {users.map(user => (
              <UserRow
                key={user.id}
                user={user}
                isEditing={editingId === user.id}
                onEdit={() => setEditingId(user.id)}
                onCancel={() => setEditingId(null)}
                onSave={(updated) => {
                  onUpdate(updated);    // UPDATE en el padre
                  setEditingId(null);   // salir de edición
                }}
                onDelete={() => onDelete(user.id)} // DELETE
              />
            ))}
          </tbody>
        </table>
      </div>
    </section>
  );
}
```

SRC/COMPONENTS/USERROW.JS

```
//src/components/UserRow.js (Fila con lectura / edición inline)
import { useEffect, useState } from 'react';

export default function UserRow({ user, isEditing, onEdit, onCancel, onSave,
onDelete }) {
  const [form, setForm] = useState(user);
  const [errors, setErrors] = useState({});

  // Resinc. del formulario cuando entra en modo edición
  useEffect(() => {
    if (isEditing) setForm(user);
  }, [isEditing, user]);

  function handleChange(e) {
    const { name, value } = e.target;
    setForm(prev => ({ ...prev, [name]: value }));
  }

  function validate(values) {
    const err = {};
    if (!values.name?.trim()) err.name = 'El nombre es obligatorio';
    if (!values.email?.trim()) err.email = 'El email es obligatorio';
    return err;
  }

  function handleSave(e) {
    e.preventDefault();
    const v = validate(form);
    setErrors(v);
    if (Object.keys(v).length > 0) return;
    onSave({ ...form, id: user.id }); // mantener id
  }

  if (!isEditing) {
    // Vista lectura
    return (
      <tr>
        <td>{user.name}</td>
        <td><a href={`mailto:${user.email}`}>{user.email}</a></td>
        <td>{user.role}</td>
        <td style={{ display: 'flex', gap: 8 }}>
          <button type="button" onClick={onEdit}>Editar</button>
          <button type="button" onClick={onDelete}>Eliminar</button>
        </td>
      </tr>
    );
  }

  // Vista edición inline
  return (
    <tr>
      <td>
        <input
          name="name"
          type="text"
          value={form.name}
```

```
        onChange={handleChange}
        aria-invalid={Boolean(errors.name)}
        aria-describedby={errors.name ? `err-name-${user.id}` : undefined}
      />
      {errors.name && <div id={`err-name-${user.id}`} style={{ color: 'crimson',
fontSize: 12 }}>{errors.name}</div>}
    </td>

    <td>
      <input
        name="email"
        type="email"
        value={form.email}
        onChange={handleChange}
        aria-invalid={Boolean(errors.email)}
        aria-describedby={errors.email ? `err-email-${user.id}` : undefined}
      />
      {errors.email && <div id={`err-email-${user.id}`} style={{ color: 'crimson',
fontSize: 12 }}>{errors.email}</div>}
    </td>

    <td>
      <select name="role" value={form.role} onChange={handleChange}>
        <option value="user">user</option>
        <option value="admin">admin</option>
      </select>
    </td>

    <td style={{ display: 'flex', gap: 8 }}>
      <button type="button" onClick={handleSave}>Guardar</button>
      <button type="button" onClick={onCancel}>Cancelar</button>
    </td>
  </tr>
 );
}
```

2.7 Ruta dinámica con React Router (UserDetail)

Una ruta dinámica contiene un parámetro de URL que cambia según el elemento (por ejemplo, un id):

- Patrón: /users/:id
- Ejemplos reales: /users/1, /users/42
- En React Router, lees ese :id con useParams() dentro del componente de detalle.

Los pasos a seguir

A. Instalar React Router (si no lo tienes)
B. Envolver tu app con <BrowserRouter> en index.js.
C. Definir las rutas en App.js:
    a. / → Home o UsersList
    b. /users → Listado de usuarios

   c. /users/:id → UserDetail (ruta dinámica)

D. Enlazar desde el listado usando <Link to={/users/${id}}>.
E. Leer el parámetro dinámico en UserDetail con useParams().
F. Obtener los datos del usuario:
   a. Opción A (recomendada): desde el estado ya cargado en App (más eficiente).
   b. Opción B: volver a fetch a users.json y filtrar por id (útil si entras directo por URL).
G. Gestionar estados: cargando, no encontrado (404), error.

SRC/INDEX.JS

```
//src/index.js (Router de alto nivel)
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

SRC/APP.JS

```
//src/App.js (Rutas + carga inicial del JSON)
import { useEffect, useState } from 'react';
import { Routes, Route, Link, Navigate } from 'react-router-dom';
import UsersList from './components/UsersList';
import UserDetail from './components/UserDetail';

export default function App() {
  const [users, setUsers]   = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError]     = useState(null);

  // Carga inicial desde public/users.json
  useEffect(() => {
    (async () => {
      try {
        const res = await fetch(`${process.env.PUBLIC_URL}/users.json`, { cache:
'no-store' });
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        setUsers(await res.json());
      } catch (err) {
        setError(err.message || 'Error cargando datos');
      } finally {
        setLoading(false);
      }
    })();
  }, []);

  return (
    <main style={{ padding: 24, fontFamily: 'system-ui, sans-serif' }}>
```

```
      <header style={{ marginBottom: 16 }}>
        <nav style={{ display: 'flex', gap: 12 }}>
          <Link to="/">Inicio</Link>
          <Link to="/users">Usuarios</Link>
        </nav>
      </header>

      <Routes>
        {/* Home */}
            <Route      path="/"      element={<p>Bienvenido.      Ve      a      <Link
to="/users">Usuarios</Link>.</p>} />

        {/* Listado */}
        <Route
          path="/users"
          element={
            loading ? (
              <p>Cargando usuarios…</p>
            ) : error ? (
              <p style={{ color: 'crimson' }}>Error: {error}</p>
            ) : (
              <UsersList users={users} />
            )
          }
        />

        {/* Ruta dinámica: /users/:id */}
        <Route
          path="/users/:id"
          element={<UserDetail users={users} loading={loading} error={error} />}
        />

        {/* Redirección o 404 */}
        <Route path="/home" element={<Navigate to="/" replace />} />
        <Route path="*" element={<p>404 — Página no encontrada</p>} />
      </Routes>
    </main>
  );
}
```

SRC/COMPONENTS/USERSLIST.JS

```
//src/components/UsersList.js(Enlaces a la ruta dinámica)
import { Link } from 'react-router-dom';

export default function UsersList({ users }) {
  if (!users?.length) return <p>No hay usuarios.</p>;

  return (
    <section aria-labelledby="users-title">
      <h2 id="users-title">Usuarios</h2>
      <ul style={{ listStyle: 'none', padding: 0, display: 'grid', gap: 8 }}>
        {users.map(u => (
          <li key={u.id} style={{ border: '1px solid #ddd', borderRadius: 8, padding:
12 }}>
            <strong>{u.name}</strong> — {u.email} <em>({u.role})</em>
            {' · '}
            {/* Enlace a la ruta dinámica */}
```

```
          <Link to={`/users/${u.id}`}>Ver detalle</Link>
        </li>
      ))}
    </ul>
  </section>
 );
}
``
```

SRC/COMPONENTS/USERDETAIL.JS

```
//src/components/UserDetail.js (Leer :id y mostrar detalle)
import { useMemo } from 'react';
import { Link, useParams } from 'react-router-dom';

export default function UserDetail({ users, loading, error }) {
  const { id } = useParams();

  // id de la URL puede venir como string → normalizamos
  const numericId = Number.isNaN(Number(id)) ? id : Number(id);

  // Buscar el usuario en el estado ya cargado
  const user = useMemo(
    () => users?.find(u => u.id === numericId || String(u.id) === String(id)),
    [users, id, numericId]
  );

  if (loading) return <p>Cargando detalle…</p>;
  if (error)   return <p style={{ color: 'crimson' }}>Error: {error}</p>;
  if (!user)   return (
    <section>
      <p>No se encontró el usuario con id <code>{id}</code>.</p>
      <Link to="/users">Volver al listado</Link>
    </section>
  );

  return (
    <section aria-labelledby="user-detail-title">
      <h2 id="user-detail-title">Detalle de usuario</h2>

      <dl>
        <dt>Id</dt><dd>{user.id}</dd>
        <dt>Nombre</dt><dd>{user.name}</dd>
                                                    <dt>Email</dt><dd><a
href={`mailto:${user.email}`}>{user.email}</a></dd>
        <dt>Rol</dt><dd>{user.role}</dd>
      </dl>

      <div style={{ marginTop: 12 }}>
        <Link to="/users">← Volver al listado</Link>
      </div>
    </section>
  );
}

/*
// Opción B (si quieres que el detalle funcione "aislado" incluso sin usuarios en
memoria):
```

```
// - Hacer fetch aquí si `users` está vacío, y gestionar un estado local.
// - Úsalo si entras directo a /users/2 con recarga.

import { useEffect, useState } from 'react';

export default function UserDetailStandalone() {
  const { id } = useParams();
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError]     = useState(null);

  useEffect(() => {
    (async () => {
      try {
        const res = await fetch(`${process.env.PUBLIC_URL}/users.json`, { cache:
'no-store' });
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        const list = await res.json();
        const found = list.find(u => String(u.id) === String(id));
        setUser(found ?? null);
      } catch (err) {
        setError(err.message || 'Error cargando detalle');
      } finally {
        setLoading(false);
      }
    })();
  }, [id]);

  if (loading) return <p>Cargando detalle…</p>;
  if (error)   return <p style={{ color: 'crimson' }}>Error: {error}</p>;
  if (!user)   return <p>No se encontró el usuario con id {id}</p>;

  return <pre>{JSON.stringify(user, null, 2)}</pre>;
}
*/
```

2.8 Simulación de persistencia (Muy importante)

En una aplicación real los datos se guardan en una base de datos y al recargar la página, los cambios se mantienen.

En una práctica con React sin backend el estado se pierde al recargar y necesitamos simular ese efecto, la solución típica es **localStorage**

El .json inicial **no es persistente**
El estado de React **no es persistente**
localStorage **sí es persistente en el navegador**

Los pasos para simular persistencia son:

JSON inicial (data.json)

```
[
  { "id": 1, "nombre": "Producto A", "precio": 10 },
  { "id": 2, "nombre": "Producto B", "precio": 20 }
]
```

Servicio de persistencia (useLocalStorageDB.js) => Este hook simula una base de datos completa.

```
import { useEffect, useState } from "react";
import dataInicial from "../data/data.json";

const KEY = "productos_db";

export function useLocalStorageDB() {
  const [data, setData] = useState([]);

  // Cargar datos iniciales
  useEffect(() => {
    const stored = localStorage.getItem(KEY);

    if (stored) {
      setData(JSON.parse(stored));
    } else {
      localStorage.setItem(KEY, JSON.stringify(dataInicial));
      setData(dataInicial);
    }
  }, []);

  // Guardar cambios
  const save = (newData) => {
    localStorage.setItem(KEY, JSON.stringify(newData));
    setData(newData);
  };

  // CRUD
  const create = (item) => {
    const newItem = { ...item, id: Date.now() };
    save([...data, newItem]);
  };

  const update = (id, updatedItem) => {
    const newData = data.map((item) =>
      item.id === id ? { ...item, ...updatedItem } : item
    );
    save(newData);
  };

  const remove = (id) => {
    const newData = data.filter((item) => item.id !== id);
    save(newData);
  };

  return { data, create, update, remove };
}
```

Uso en un componente React

```
import { useLocalStorageDB } from "./hooks/useLocalStorageDB";

export default function App() {
  const { data, create, update, remove } = useLocalStorageDB();

  return (
    <div>
      <h1>Productos</h1>

      <button onClick={() => create({ nombre: "Nuevo", precio: 99 })}>
        Añadir producto
      </button>

      <ul>
        {data.map((p) => (
          <li key={p.id}>
            {p.nombre} - {p.precio}€
            <button onClick={() => update(p.id, { precio: p.precio + 1 })}>
              +1€
            </button>
            <button onClick={() => remove(p.id)}>Eliminar</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

Esto le permite entender el flujo completo **sin necesidad de servidor**

CONCEPTO	SIMULACIÓN
Backend real	localStorage
Base de datos	JSON inicial + actualizaciones
API CRUD	Funciones créate/update/remove
Persistencia	localStorage.setItem